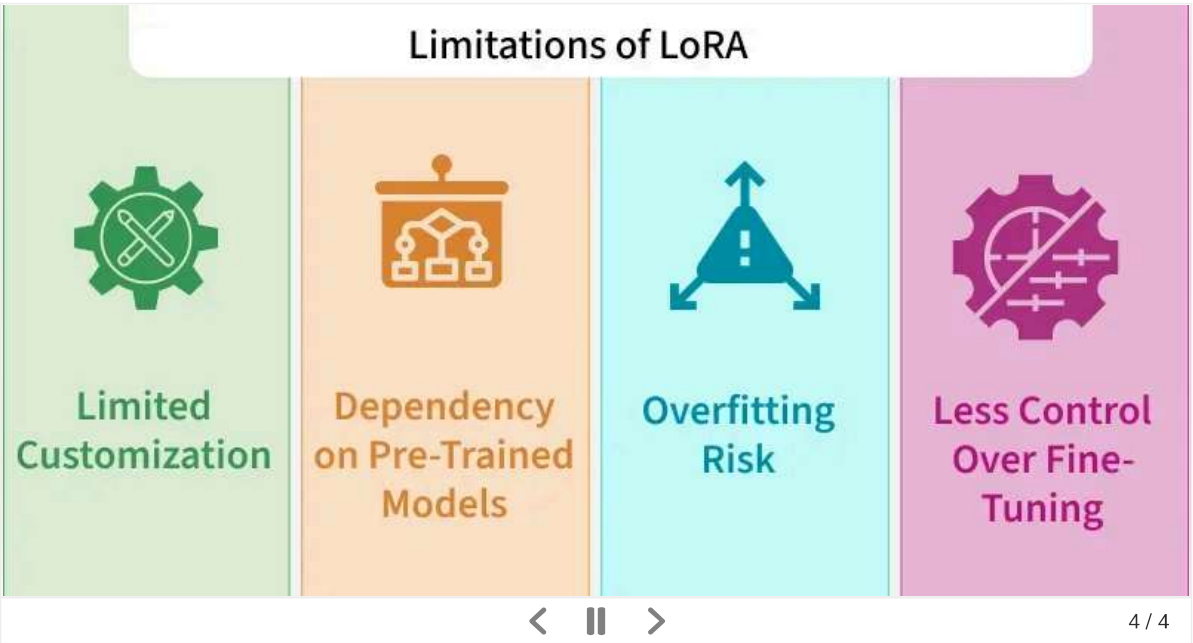
What is Low Rank Adaptation (LoRA)?

Last Updated : 15 Apr, 2026

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning technique used to adapt large pre-trained models for specific tasks with minimal computational and memory overhead. As models grow larger, full fine-tuning becomes expensive. LoRA addresses this by reducing the number of trainable parameters, making the process faster and more efficient.



Instead of updating the entire model, it introduces low-rank matrices into the model's layers. This allows us to adapt the model for specific tasks without changing the whole structure, maintaining high performance while reducing resource usage.

Key Features of LoRA

1. **Parameter Efficiency**: It reduces the number of trainable parameters, leading to lower memory usage during fine-tuning and inference.
2. **Computational Efficiency**: It minimizes matrix operations, reducing the computational workload on GPUs/TPUs and speeding up the fine-tuning process.
3. **Preservation of Pre-Trained Knowledge**: The original pre-trained model remains unchanged making it easy to revert to the base model when needed.
4. **Scalability**: It can be applied to various transformer-based models like GPT, BERT and T5 making it versatile for different tasks.
5. **Faster Fine-Tuning**: By updating fewer parameters, it accelerates the fine-tuning process compared to traditional methods.

Architecture of LoRA

LoRA is used with transformer-based models which are common in NLP tasks. Let's see how it works:

1. **Pre-Trained Backbone**: We start with a large transformer model like GPT or BERT that has already been trained on a range of data.

2. **Low-Rank Adaptation Layers**: It adds small low-rank matrices to the model's attention mechanism. These matrices are the only parts of the model that get updated during fine-tuning.
3. **Frozen Original Parameters**: The original weights of the model are kept frozen. This means we don't modify the entire model, just the added low-rank matrices.
4. **Task-Specific Fine-Tuning**: We fine-tune the low-rank matrices for the specific task such as sentiment analysis or translation while the rest of the model stays the same.

This approach helps us adapt large models to new tasks without changing the entire structure making it more efficient.

Working of LoRA

LoRA modifies the traditional fine-tuning process by introducing low-rank matrices into specific layers of a neural network allowing the model to adapt to new tasks without changing the entire model. Let's see how LoRA works:

1. Decomposing the Weight Matrix

Instead of updating the entire weight matrix during fine-tuning, it approximates it using two smaller low-rank matrices A and B. The adapted weight matrix (W') is calculated as:

$$W' = W + A \cdot B$$

Here W is the original weight matrix and A and B are the low-rank matrices. This decomposition allows the model to make task-specific adjustments without the need to retrain the entire model, drastically reducing the computational load.

2. Training Only the LoRA Parameters

During the fine-tuning process, only the low-rank matrices A and B are updated while the original model weights W remain frozen. This minimizes the number of parameters that need to be adjusted making fine-tuning faster and more memory-efficient compared to traditional methods where all model weights are updated.

3. Inference with Adapted Weights

After fine-tuning, the adapted weight matrix W' is used for inference. This helps the model to make predictions for specific tasks, fine-tuned with minimal computational resources. Since only the low-rank matrices are updated, it maintains efficiency even during inference.

By using LoRA, we can adapt large pre-trained models to new tasks quickly and efficiently without the computational burden of full model fine-tuning.

Implementation of LoRA with BERT on Emotion Detection

Here we will see a practical implementation of LoRA on the Emotion Dataset. Instead of updating the entire BERT model, we fine-tune only small LoRA modules, saving time and resources while still achieving good performance in classifying emotions such as joy, sadness, anger, love, fear and surprise.

1. Installing Required Libraries

We use Hugging Face's transformers, datasets, peft, accelerate and evaluate libraries for model training, LoRA fine-tuning and evaluation.

!pip install transformers datasets peft accelerate evaluate

2. Importing Dependencies

We will be importing BERT, tokenizer, LoRA config, dataset loader, training utilities and PyTorch for this implementation.

```
from transformers import AutoModelForSequenceClassification, AutoTokenizer, TrainingArgum
Trainer, pipeline
from datasets import load_dataset
from peft import LoraConfig, get_peft_model
import evaluate
import torch
```

3. Loading the Dataset

We load the Emotion dataset(dair-ai/emotion). For quick implementation, we use only 3,000 training samples and smaller validation/test subsets.

```
dataset = load_dataset("dair-ai/emotion")
dataset["train"] = dataset["train"].shuffle(seed=42).select(range(3000))
dataset["validation"] = dataset["validation"].shuffle(seed=42).select(range(500))
dataset["test"] = dataset["test"].shuffle(seed=42).select(range(500))
```

Output:

```
README.md: 9.05k/? [00:00<00:00, 792kB/s]
split/train-00000-of-00001.parquet: 100% 1.03M/1.03M [00:00<00:00, 1.31MB/s]
split/validation-00000-of-00001.parquet: 100% 127k/127k [00:00<00:00, 21.6kB/s]
split/test-00000-of-00001.parquet: 100% 129k/129k [00:00<00:00, 234kB/s]
Generating train split: 100% 16000/16000 [00:00<00:00, 7926.40 examples/s]
Generating validation split: 100% 2000/2000 [00:00<00:00, 38052.03 examples/s]
Generating test split: 100% 2000/2000 [00:00<00:00, 42442.37 examples/s]
```

Loading the Dataset

4. Preprocessing Data

Before training, we tokenize the text so BERT can process it. Each input is padded or truncated to 128 tokens for uniformity. We also rename label to labels and set the dataset format to PyTorch tensors. Then we load BERT-base-uncased for sequence classification with 6 output labels.

```
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)

def preprocess(batch):
    return tokenizer(batch["text"], truncation=True, padding="max_length", max_length=128)

dataset = dataset.map(preprocess, batched=True).rename_column("label", "labels")
dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])
model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=6)
```

Output:

```
tokenizer_config.json: 100%|| 48.0/48.0 [00:00<00:00, 686B/s]
config.json: 100%|| 570/570 [00:00<00:00, 10.8kB/s]
vocab.txt: 100%|| 232k/232k [00:00<00:00, 3.52MB/s]
tokenizer.json: 100%|| 466k/466k [00:00<00:00, 9.37MB/s]
Map: 100%|| 3000/3000 [00:01<00:00, 1624.14 examples/s]
Map: 100%|| 500/500 [00:00<00:00, 3152.06 examples/s]
Map: 100%|| 500/500 [00:00<00:00, 2717.87 examples/s]
model.safetensors: 100%|| 440M/440M [00:05<00:00, 126MB/s]
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

Preprocessing Data

5. Applying LoRA Configuration

Instead of training all of BERT's 110M+ parameters, LoRA injects small trainable matrices into the attention layers (query, value). This makes fine-tuning efficient without sacrificing much performance.

```
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["query", "value"],
    lora_dropout=0.1,
    bias="none",
    task_type="SEQ_CLS"
)
model = get_peft_model(model, lora_config)
```

6. Training the Model

We fine-tune only the LoRA layers using Hugging Face's Trainer API.

- Batch size = 8
- Learning rate = 2e-4 (slightly higher since only LoRA layers are trained)
- Epochs = 2 (kept short for quick results)

```
metric = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = logits.argmax(axis=-1)
    return metric.compute(predictions=preds, references=labels)

args = TrainingArguments(
    output_dir="./lora_emotion",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    learning_rate=2e-4,
    num_train_epochs=2,
    eval_strategy="epoch",
    logging_steps=10,
    report_to="none"
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"],
    compute_metrics=compute_metrics
)
```

```
trainer.train()
model.save_pretrained("./lora_emotion_adapter")
print("LoRA fine-tuning complete! Adapter saved at ./lora_emotion_adapter")
```

Output:

Downloading builder script: 4.20k/? [00:00<00:00, 134kB/s]

[750/750 01:52, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Accuracy
1	1.287800	1.243594	0.542000
2	1.121900	1.146290	0.554000

```
LoRA fine-tuning complete! Adapter saved at ./lora_emotion_adapter
```

Training the Model

The accuracy here is not very high (~54%) because we trained only for 2 epochs on a small subset of the dataset. This setup is mainly for demonstration and understanding. For better performance, we can train for more epochs and use the full dataset.

7. Testing the Model on Custom Sentences

Now let's test our fine-tuned model on some custom sentences. This helps us confirm that the LoRA adapter works as intended.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

emotion_labels = dataset["train"].features["labels"].names

samples = [
    "I am so happy to see you!",
    "This is terrifying, I can't handle it.",
    "He surprised everyone with his gift.",
    "I feel so sad and lonely.",
    "I love spending time with my family."
]

inputs = tokenizer(samples, truncation=True, padding=True, return_tensors="pt").to(device)

model.eval()
with torch.no_grad():
    logits = model(**inputs).logits
    preds = torch.argmax(logits, dim=-1)

for text, pred in zip(samples, preds):
    print(f"Text: {text}\nPredicted Emotion: {emotion_labels[pred]}\n")
```

Output:

```
Text: I am so happy to see you!
Predicted Emotion: joy

Text: This is terrifying, I can't handle it.
Predicted Emotion: sadness

Text: He surprised everyone with his gift.
Predicted Emotion: joy

Text: I feel so sad and lonely.
Predicted Emotion: sadness

Text: I love spending time with my family.
Predicted Emotion: joy
```

Testing the Model

LoRA vs Other Fine-Tuning Techniques

Different fine-tuning techniques have their strengths and weaknesses. Let's see a comparison of LoRA with some of the common techniques used for fine-tuning large models:

Technique	Parameter Efficiency	Computation Cost	Model Preservation
Full Fine-Tuning	High	High	No
Adapter Layers	Moderate	Moderate	Yes
LoRA	High	Low	Yes
Prefix Tuning	High	Moderate	Yes

Applications of LoRA

LoRA is utilized across multiple domains, particularly when fine-tuning large pre-trained models. Let's see some key applications:

1. **Natural Language Processing (NLP)**: It is used to fine-tune large transformer models like GPT, BERT and T5 for various tasks including text classification, sentiment analysis, summarization and machine translation.
2. **Computer Vision**: It can be applied to vision transformers (ViTs) for tasks such as image classification, object detection and segmentation, enabling efficient adaptation to specific image-related tasks.
3. **Speech Processing**: It is also effective in speech recognition tasks such as automatic speech recognition (ASR) and text-to-speech (TTS) systems making it easier to adapt these models to specific languages or accents.
4. **Large Language Models (LLMs)**: It is used to fine-tune large models like GPT-3, GPT-4 and LLaMA, reducing the computational overhead and enabling efficient task adaptation without the need for costly retraining.

Advantages of LoRA

LoRA has several key benefits for fine-tuning large models which includes:

1. **Reduced GPU Memory Consumption**: By only updating low-rank matrices, it reduces memory usage making it easier to fine-tune large models on hardware with limited resources.
2. **Faster Fine-Tuning**: It speeds up the fine-tuning process by updating fewer parameters resulting in faster task-specific adaptation compared to traditional full fine-tuning.
3. **Improved Transferability**: Since the original model weights remain frozen, it allows easy adaptation to multiple tasks, preserving the knowledge from the pre-trained model.
4. **Supports Multi-Task Learning**: It enables training multiple low-rank adapters for different tasks without interference, supporting efficient multi-task learning.

Limitations of LoRA

While LoRA helps for efficient fine-tuning, there are a few limitations to keep in mind:

1. **Task-Specific Limitations**: It performs best when the pre-trained model is already well-aligned with the target task. For tasks that differ from the model's original training, it may not be as effective as full model fine-tuning.

2. **Complexity in Selection**: Deciding where to apply the low-rank matrices within the model such as which layers to target, requires careful consideration and expertise. An improper selection could result in suboptimal performance.
3. **Model Size**: For smaller models, the benefits of using LoRA may not be as noticeable since the extra cost of fine-tuning the entire model is not that high. In these cases, full fine-tuning might be just as effective without needing LoRA.

You can download source code from here.

Related Articles:

- *What is Parameter-Efficient Fine-Tuning (PEFT)?*
- *Fine-Tuning using LoRA and QLoRA*

Comment

S sanjuli... 4

Article Tags: Deep Learning AI-ML-DS

Explore

Basics

Neural Networks

Deep Learning Models

Model Evaluation

Deep Learning Frameworks

Projects

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company
About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Training Program

Explore
POTD
Job-A-Thon
Blogs
Nation
Skill Up

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos
DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

GET IT ON Google Play
Download on the App Store

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved